

Q1 代码架构与运行逻辑完全指南

VLSI 布图规划设计 — 自由轮廓面积最小化（面积 + 长宽比）

2026 年第七届” 华数杯” 大学生数学建模竞赛 B 题

2026/08

目录

1	项目总体架构	2
1.1	目录结构	2
1.2	执行顺序	3
1.3	核心设计理念	3
2	Phase 1: B*-Tree 核心 floor_plan.hpp	3
2.1	功能概述	3
2.2	关键结构	3
2.3	输入格式	4
3	Phase 2: 快速模拟退火引擎 sa.hpp	4
3.1	Mode 参数化	4
3.2	代价函数	4
3.3	收敛 log 与过程快照	5
4	Phase 3: CLI main.cpp	5
5	Phase 4: 求解驱动与可视化	5
5.1	q1_solver.py	5
5.2	每芯片 10+1 张图	5
6	测试套件 test_main.cpp (1027 断言)	6
6.1	oracle 体系	6
6.2	运行	6

7 关键设计决策	6
7.1 为什么共享核心而不是每问复制?	6
7.2 为什么 $\lambda = 0.5$?	6
7.3 为什么初始温度 /50 加热到 /20?	7
7.4 为什么多轮取优?	7
7.5 为什么差分测试而不只靠单元测试?	7
8 实际运行结果	7
8.1 指标汇总 ($\lambda = 0.5$, 5 轮取优, Python 独立复核)	7
8.2 可视化	7
9 文件依赖关系	7
10 运行指南	8
10.1 环境要求	8
10.2 完整运行流程	8
10.3 常见问题	8
11 合规清单	8

1 项目总体架构

1.1 目录结构

```
1 The_7th_Huashu_Cup.../
2 |-- data/
3 |   |-- raw/                # 原始附件副本 n100/n200/n300 (只读)
4 |-- cpp_solver/            # C++ 共享核心 (Q1/Q2/Q3 共用, 单套测试)
5 |   |-- Makefile            # make / make test / make test-san
6 |   |-- src/
7 |       |-- floor_plan.hpp  # B*-Tree 拓扑 + Skyline 解码 + 解析器
8 |       |-- sa.hpp          # 快速模拟退火引擎 (Mode 参数化 + 收敛 log + 快照)
9 |       |-- main.cpp        # CLI 入口 (mode/alpha/--log)
10 |      |-- test_main.cpp    # 白盒测试 (1027 断言, 含 oracle 体系)
11 |      |-- test_oracle.hpp  # 独立参考解码器 / 合法性校验 / 暴力枚举
12 |-- common/
13 |   |-- visualize.py        # 共享绘图基础 (.rpt/.log/snap 解析 + 布图绘制)
14 |-- q1/                    # 问题 1 客制层 (<-- 本文件)
15 |   |-- q1_solver.py        # 求解驱动: 调 bin/main, 多轮取优, 汇总 CSV
16 |   |-- output/
17 |       |-- q1_metrics.csv  # 三芯片指标汇总 (面积/长宽比/利用率/合法性)
18 |       |-- q1_n100.rpt     # 最终布局坐标
19 |       |-- q1_n100.log     # 收敛轨迹 + 9 等分过程快照
20 |   |-- visualization/
21 |       |-- plot_floorplan.py # 最终布图
22 |       |-- plot_convergence.py # 收敛曲线
23 |       |-- plot_snapshots.py # 9 等分过程快照 (每芯片 9 张)
24 |       |-- figs/<时间戳>/  # 每次运行新建时间戳目录 (保留全部历史)
25 |           |-- <chip>/    # 每芯片 11 张图: 9 快照 + 收敛 + 最终布图
26 |-- q2/ q3/ q4/            # 其余问题 (共享核心 + 各自客制层)
```

1.2 执行顺序

执行顺序

1. `cd cpp_solver && make` (编译 `bin/main`, 0 警告)
2. `make test` (可选, 白盒测试回归, 1027 断言)
3. `conda activate math`
4. `python q1/q1_solver.py` (求解三芯片 + 多轮取优 + 出图)

1.3 核心设计理念

共享核心 + 每问客制层: B*-Tree 拓扑、Skyline 解码、SA 引擎、解析器、测试 oracle 是 Q1/Q2/Q3 共用的”已验证机械资产”, 只维护一套、单套测试回归 (避免复制陷阱)。每问的模型表达 (代价函数、可行性判定、阶段策略) 独立成客制层, 论文手可直接读到模型。

差分测试方法论: 正确性不靠”测试通过”, 靠独立实现的 oracle 交叉验证 (参考 skyline 解码器、独立 HPWL、暴力枚举最优、定义校验器), 并辅以 ASan/UBSan 双轨。

2 Phase 1: B*-Tree 核心 floor_plan.hpp

2.1 功能概述

封装 B*-Tree 表示、三种扰动算子 (旋转 / 删除-插入 / 节点交换)、Skyline 轮廓解码、竞赛格式解析器 (`.blocks/.nets/.pl`)。

2.2 关键结构

- `NODE`: 10-bit 位压缩的父子/左右子指针 + 旋转位 (内存池, 支持至 1023 节点)。
- `TREE`: 拓扑操作 (`del_from_tree` / `ins_to_tree` / `swap_two_nodes` / `swap_near` / `swap_not_near`), 均通过 `ValidateTree()` 白盒校验 (DFS 计数 == `N`、无环、父子双向绑定)。
- `init()`: DFS 前序放置, `find_max_y` 维护 `y` 轮廓 (`std::list` + 迭代器前插语义), 天然无重叠。
- `cost()`: 返回 (HPWL, `W`, `H`); HPWL 按引脚几何中心 (块: $2x + w$, 终端: $2x$)。

- 构造器 `free_outline` 参数: Q1 模式跳过”超轮廓强制旋转” (自由轮廓无此概念)。

2.3 输入格式

```
1 .blocks: NumHardBlocks : 100 / NumTerminals : 334 / b0 block 4 (0,0)
    (0,33) (43,33) (43,0)
2 .nets: NumNets : 885 / NetDegree : 2 / p1 / b26
3 .p1: p1 0 0
```

3 Phase 2: 快速模拟退火引擎 sa.hpp

3.1 Mode 参数化

```
1 enum class Mode { Q1, Q2 }; // Q1: 自由轮廓面积+长宽比; Q2: 固定轮廓+HPWL
```

Q1 模式 (本问):

- `feas()` 恒真 (无轮廓约束)。
- `q1_cost`: 加权代价 (见 3.2), 固定权重, 无自适应 β 压力机制。
- 单阶段: 仅 `run2` (精修退火), 不做 Q2 的”先找可行”阶段。
- 初始温度加热: `_init_T2 = _init_T / 20` (Q2 为 /50), 提升探索性。

3.2 代价函数

$$f = \lambda \cdot \frac{W \cdot H}{\bar{A}} + (1 - \lambda) \cdot \frac{r + 1/r - 2}{\bar{P}}, \quad r = \frac{\max(W, H)}{\min(W, H)} \quad (1)$$

- 长宽比惩罚采用对称形式 $r + 1/r - 2$: 旋转整个芯片等效于长宽比取倒数, 对称惩罚保证两者代价相同。
- $\lambda = 0.5$: 面积与长宽比归一化后均衡 (经扫描 $\lambda \in [0.45, 0.95]$ 确定)。
- $\bar{A}, \bar{P}$: 构造期采样 $N + 1$ 个随机布局的归一化均值。

3.3 收敛 log 与过程快照

- 每温度层写一行: `phase iter T best_cost alpha feas`。
- 每个 `run2` 结束后按温度层数 9 等分输出 9 个 checkpoint 布局: `snap k iter W H 头 + N 行 name x1 y1 x2 y2`。
- 画图程序与求解完全解耦: 求解只写文件, 可视化后处理读取 log 出图。

4 Phase 3: CLI main.cpp

```
1 ./bin/main <mode:q1|q2> <alpha> <blocks> <nets> <pl> <rpt> [  
    dead_ratio] [--log <file>]
```

- `<alpha>` 即 λ (Q1 模式面积权重)。
- `[dead_ratio]`: Q2 模式的正方形轮廓参数 (Q1 忽略)。
- `--log`: 收敛轨迹 + 快照输出 (Q1 模式由 `q1_solver.py` 传入)。
- 输出 `.rpt`: `cost / hpwl(=0) / area / W H / time / 每模块坐标`。

5 Phase 4: 求解驱动与可视化

5.1 q1_solver.py

1. 三芯片并行调用 `bin/main q1` (每芯片 `--repeats 5` 轮取 `(area, aspect)` 字典序最优, 对抗 SA 随机波动)。
2. 独立复核: 无重叠 / 尺寸保持 / 面积 $\geq \sum$ 块面积 (Python 独立实现, 不信任 C++)。
3. 汇总 `q1_metrics.csv`。
4. 画图: `figs/<YYYYMMDD_HHMMSS>/<chip>/` (每次运行新建时间戳目录, 保留全部历史)。

5.2 每芯片 10+1 张图

- `snap_01.png ... snap_09.png`: SA 过程 9 等分快照 (初始 $\rightarrow$ 最终的布局演化, 标题含层号)。
- `convergence.png`: `best cost` 收敛曲线 (归一化)。

- floorplan.png: 最终布图 (面积/长宽比标注)。

6 测试套件 test__main.cpp (1027 断言)

6.1 oracle 体系

- **O1 定义校验**: 从 B*-Tree 定义出发验证 $\times$ 关系 + 原点 + 无重叠 + 尺寸保持。
- **O2 差分解码**: 独立参考解码器 (数组 skyline + 严格相交语义, 与实现完全不同算法), 随机树/全部形状逐块全等; 穷举 $N=6/7/8$ 全部二叉树形状 (Catalan 132/429/1430) $\times$ 多组标签/旋转。
- **O3 cost oracle**: 独立 HPWL (无预处理/无排序) 与 `cost()` 全等; 手算用例 (含旋转后中心映射)。
- **O4 算子语义**: 交换 = 位置互换且节点集合并集守恒; 删除-插入子树不丢失。
- **O5 SA**: 固定种子可复现、best 历史单调 (分段)、与暴力枚举最优对照。
- **D 系列**: 内存安全 ($N=1$ 除零守卫、1023 节点压测、Crusher 500 模块)、Skyline 极限、SA 数值 (NaN 防护)、解析器三格式。

6.2 运行

```
1 cd cpp_solver && make test      # 1027 断言
2 make test-san                  # ASan + UBSan 双轨 (0 告警)
```

7 关键设计决策

7.1 为什么共享核心而不是每问复制?

B*-Tree/解码/SA 是 Q1-Q3 共用的机械正确性资产 (已由 1027 断言证明); 复制会造成 4 份代码 4 倍测试与 4 倍修 bug 成本 (复制陷阱)。模型差异 (代价/约束/阶段) 通过 Mode 参数化 + 每问客制层表达。Q4 (L/T 模块) 是不同模型, 独立开发但复用测试方法论。

7.2 为什么 $\lambda = 0.5$?

扫描 $\lambda \in [0.45, 0.95]$ 得到权衡曲线: $\lambda = 0.8$ 时 SA 卡在长条局部最优 (n300 长宽比 4.4); $\lambda = 0.5$ 时 n100 达到长宽比 1.002、面积利用率 0.942。0.5 是面积与长宽比的均衡点 (归一化后两项同量级)。

7.3 为什么初始温度 /50 加热到 /20?

HW2 原版 /50 对 Q1（无可行性压力）太冷，搜索易陷局部最优；加热后 n100 长宽比从 1.9 降到 1.0，且面积利用率反而提升。

7.4 为什么多轮取优?

SA 是随机启发式，单次运行结果波动 (n300 长宽比 1.5 4.4); 5 轮取 (area, aspect) 字典序最优可将波动压到可接受范围。

7.5 为什么差分测试而不只靠单元测试?

白盒测试的期望值若来自实现推导会”被代码绑架”（本项目曾两次手推错误）。独立参考解码器 + 穷举形状覆盖，从语义定义出发验证，才是真正的正确性证据。

8 实际运行结果

8.1 指标汇总 ($\lambda = 0.5$, 5 轮取优, Python 独立复核)

芯片	轮廓 $W \times H$	面积	长宽比	面积利用率	耗时 (s)	合法性
n100	421×455	191555	1.081	0.937	0.85	合法
n200	357×517	184569	1.448	0.952	15.9	合法
n300	486×591	287226	1.216	0.951	43.9	合法

- 面积利用率 = 模块总面积 / 轮廓面积，均 > 93% (B*-Tree 自由轮廓典型水平)。
- 长宽比全部满足验证标准 (n100/n200 ≤ 1.5 , n300 ≤ 1.5)。
- 三组芯片全部通过独立合法性复核（无重叠、尺寸保持）。

8.2 可视化

每个芯片 11 张图：9 张 SA 过程快照（初始到最终的布局演化）、1 张收敛曲线、1 张最终布图。输出位于 q1/visualization/figs/< 时间戳 >/<chip>/。

9 文件依赖关系


```

1 q1_solver.py
2 |-- cpp_solver/bin/main          (编译自 main.cpp + sa.hpp +
   floor_plan.hpp)
3 |-- data/raw/{n100,n200,n300}.{blocks,nets,pl}
4 |-- common/visualize.py
5 |-- q1/visualization/plot_{floorplan,convergence,snapshots}.py

```

10 运行指南

10.1 环境要求

- g++ (Apple clang, `-std=c++17`, 无第三方依赖)。
- conda 环境 `math` (Python 3.9+, matplotlib)。
- TeX Live 2026 (xelatex + ctex, 编译本指南)。

10.2 完整运行流程

```

1 cd cpp_solver && make && make test      # 编译 + 白盒测试
2 cd ..
3 conda activate math
4 python q1/q1_solver.py                  # 求解 + 汇总 + 出图
5 python q1/q1_solver.py --skip-solve     # 只重出图/汇总 (复用已有结
   果)

```

10.3 常见问题

1. 图没更新: 求解只写 `output/`, 画图每次新建时间戳目录——直接看最新目录。
2. 长宽比不达标: 增大 `--repeats` 或调低 `--alpha`。
3. 性能: `n300`单轮约 15s (并行三芯片); 求解与画图解耦, 不互相拖累。

11 合规清单

- 原始数据 `data/raw/` 为只读副本, 求解器不修改输入文件。
- 全部结果可通过 `make test` (1027 断言) + `make test-san` 复现。

- 随机种子未固定（每次求解不同），论文数值以 `q1_metrics.csv` 与 `figs/< 时间戳 >/` 为准；复现流程确定性（命令固定）。
- 核心代码与 HW2 开源方案的差异已在转换阶段记录（去 `gnuplot/Boost`、解析器适配、NaN 除零修复、N=1 守卫、reset 上限等）。